TECHNICAL DEFENSE & INTERVIEW PREPARATION GUIDE

Hierarchical Federated Learning for Privacy-Preserving IoT Intrusion Detection Using Unsupervised Anomaly Detection

Comprehensive Q&A; Defense Manual covering Architectures, Machine Learning Engineering Decisions, Preprocessing Leakages, and System Debugging

Prepared For: Capstone Project Presentation and Technical Review Panels

Author Placeholder: **[Information Needed]**

Institution Placeholder: **[Information Needed]**

Date: July 2026

1. Project Overview & Architectural Foundation

Q1: Give me a brief overview of your IoT project.

A: This project implements a privacy-preserving network intrusion detection system (NIDS) simulated across a 5-layer IoT hierarchy (Edge, Gateway, Fog, Proxy, and Cloud). Local Edge nodes train a Mini Transformer Autoencoder in an unsupervised anomaly detection paradigm using only benign network flow records. Updates are relayed bottom-up and aggregated using Federated Averaging (FedAvg) at the Fog and Cloud levels. Anomalies are flagged at inference when reconstruction error (MSE) exceeds a dynamic threshold. Finally, a SHAP explainability module identifies which header features drive anomaly attributions.

Q2: What problem were you trying to solve?

A: Traditional NIDS requires sending raw network logs to the cloud. This incurs massive bandwidth overhead, high transmission latency, and major privacy violations. Additionally, standard centralized machine learning models suffer from: (1) target leakages where ground-truth indicators cheat during training, (2) zero-support data splits that produce deceptive evaluation accuracies, and (3) a high rate of false positive alarms due to rigid thresholds.

Q3: Why did you choose this project?

A: This project combines several critical aspects of modern computer engineering: decentralized systems, deep learning sequence models, network protocol security, data pipeline integrity, and explainable AI (XAI). It represents a highly practical approach to securing sensitive enterprise IoT installations under realistic bandwidth limitations.

Q4: Can you explain the architecture from Edge to Cloud?

A: The system consists of five distinct logical layers: (1) **Edge/Sensors** (Sensor 1 & 2) perform preprocessing, local autoencoder training, and float16 model updates. (2) **Gateway Node** (Bridge 1) buffers weights from both edges in a dictionary key structure to avoid overwriting. (3) **Fog Layer** performs regional weight aggregation using FedAvg. (4) **Proxy Node** acts as a transparent network relay. (5) **Cloud Node** aggregates the final global weights and broadcasts updates back down the hierarchy.

Q5: Walk me through the data flow from start to finish.

A: The pipeline flows as follows: Preprocessing isolates the target column 'Label', drop the ground-truth 'Attack_categories', scale features, and execute StratifiedKFold into B.1/B.2/B.3 test splits. Train Part A is split into A.1 (Sensor 1) and A.2 (Sensor 2). Sensors filter out malicious traffic, keeping only benign samples, and train the Autoencoder. Trained weights are quantized to float16, relayed by Gateway, and aggregated at Fog and Cloud using FedAvg. The Cloud Node determines a dynamic anomaly threshold: Mean(MSE) + 4*Std(MSE) on normal test records. Test splits are reconstructed, flagging anomalies if their reconstruction MSE exceeds the threshold. The App node then runs SHAP explainability on flagged flows.

Q6: What is your role in this project?

A: I designed and implemented the entire project from scratch. This includes formulating the Mini Transformer Autoencoder model, implementing the hierarchical network simulation nodes, developing the leak-free data preprocessing pipeline, writing the stratified partitioning logic, resolving weight overwriting bugs in the Gateway, implementing float16 quantization, and writing the ReportLab PDF compilation scripts.

Q7: What would happen if one Edge node fails?

A: Because the Gateway buffers updates in a key-value dictionary, the system survives the failure of a client. If Sensor 1 goes offline, Bridge 1 forwards only Sensor 2's weights. The Fog node aggregates updates from the active client. This demonstrates the decentralized fault-tolerance of Federated Learning architectures.

2. Federated Learning & Weight Aggregation

Q1: Why Federated Learning instead of centralized learning?

A: Centralized NIDS is a privacy hazard that exposes sensitive local IP headers and payloads. It also creates a massive bandwidth bottleneck when uploading millions of flow records. Federated Learning transmits only model weight adjustments, leaving raw user traffic isolated on local devices.

Q2: What problem does Federated Learning solve?

A: It solves the privacy-utility trade-off. Organizations can benefit from a shared global model trained on massive, diverse datasets without compromising user data privacy or incurring prohibitive WAN data transfer costs.

Q3: What is FedAvg?

A: Federated Averaging (FedAvg) is the standard algorithm for aggregating model updates. It takes the weight matrices from decentralized clients, scales them proportionally based on the local sample size, and performs an element-wise arithmetic average to compile a new global weight dictionary.

Q4: How does weight aggregation work?

A: Weights are aggregated by extracting state dicts from the client models. For each parameter tensor key, the aggregator performs: global_tensor = sum(client_tensors) / N. The resulting aggregated state dict is then loaded back into the global model.

Q5: Why not send raw data?

A: Raw packet logs contain unencrypted payloads, credentials, and network mapping indicators that violate corporate privacy policies. Furthermore, transmitting raw data consumes heavy network bandwidth, whereas transmitting model weights is a fixed, predictable, and highly compressible payload.

Q6: What happens during one communication round?

A: 1. Cloud broadcasts current global weights. 2. Edge clients pull global weights and load them. 3. Edge clients train locally for 5 epochs on benign data. 4. Clients extract weights, cast to float16, and upload to the Gateway. 5. Gateway buffers updates, and Fog averages them. 6. Aggregated weights are passed up to Cloud to establish the final global weights.

Q7: Why did you simulate multiple layers?

A: Industrial IoT systems are naturally hierarchical. Direct client-to-cloud updates suffer from high latency and WAN packet loss. Simulating a 5-layer topology replicates realistic enterprise routing where regional Gateways and Fog nodes aggregate local clusters before sending summary checkpoints to the cloud.

Q8: If there were 100 Edge nodes instead of 2, what changes?

A: The Gateway buffer must scale from 2 to 100 dictionary slots. To handle uneven client datasets, we would implement weighted FedAvg: weight_global = sum(n_k * weight_k) / sum(n_k), where n_k is the number of benign packets trained at client k.

Q9: What communication bottlenecks exist?

A: Network latency, asymmetric cellular channels (slow upload speeds), and client dropouts ('stragglers') who stall the aggregation loop by failing to complete local training rounds in time.

Q10: How would Secure Aggregation improve your project?

A: Secure Aggregation (SecAgg) uses cryptographic protocols (like secret sharing or homomorphic encryption) to combine client updates such that the aggregator can only decrypt the averaged sum, preventing the coordinator from inspecting individual weights, which eliminates potential model-poisoning or model-inversion privacy threats.

3. Deep Learning & Sequence Reconstruction

Q1: Why Transformer?

A: Transformers leverage self-attention mechanisms to map dependencies across time steps without relying on sequential recurrent steps. This allows the model to capture complex temporal interactions between different network packets in a sliding window much more effectively.

Q2: Why not CNN?

A: Convolutional Neural Networks (CNNs) assume local spatial grid structures (e.g. adjacent pixels). They are not natively designed to capture long-range temporal correlations across sequence windows in network protocol streams.

Q3: Why not LSTM?

A: Long Short-Term Memory (LSTM) models process sequences step-by-step, preventing parallel training. They also suffer from information bottlenecks and vanishing gradients over longer seq_len inputs, whereas self-attention enables direct connections between any two packets in a sequence.

Q4: Why Autoencoder?

A: Autoencoders compress input sequences into a low-dimensional latent space and attempt to reconstruct the original input. By training the network exclusively on normal traffic, it learns to reconstruct normal traffic with low error. Anomalous traffic, having features unseen during training, cannot be compressed and reconstructed accurately, creating a high reconstruction error that serves as an anomaly indicator.

Q5: Why an unsupervised approach?

A: Supervised models are unable to detect zero-day attacks because they require labeled examples of every attack class to train. Unsupervised models learn the boundary of normal behavior and flag any deviation, enabling them to detect novel, unseen security threats.

Q6: What is reconstruction error?

A: It is the Mean Squared Error (MSE) between the input feature vector sequence and the model's output reconstructed sequence. It measures how much the input packet sequence deviates from the normal baseline patterns the model has learned.

Q7: What loss function are you using?

A: PyTorch's `nn.MSELoss()`, which calculates the squared Euclidean distance between the input and reconstructed tensors.

Q8: Why only train on benign traffic?

A: If the autoencoder trained on attack traffic, it would learn to reconstruct attacks with low error. This would eliminate the reconstruction error difference between normal and anomalous packets, making the model blind to intrusions.

Q9: How is anomaly detection performed?

A: A threshold is calculated from normal validation sequences: Threshold = Mean(MSE) + 4 * Std(MSE). During testing, reconstruction MSE is calculated. If the MSE exceeds the threshold, the packet sequence is flagged as an attack (1); otherwise, it is normal (0).

Q10: What is the latent representation?

A: The compressed vector bottleneck output by the encoder (dimension = 32). It represents the most critical low-dimensional temporal signatures of the packet sequence, discarding noise and high-frequency fluctuations.

4. Data Engineering & Integrity

Q1: Why remove Attack_categories?

A: The column `Attack_categories` contains string values like 'DDoS' or 'normal'. When one-hot encoded, it creates features like `Attack_categories_normal` (which is exactly equivalent to `1 - Label`). This column leaks the ground truth label directly into features, allowing the model to cheat during training.

Q2: What is feature leakage?

A: Feature leakage occurs when training features contain direct information about the target label that would not be available at inference. It artificially inflates validation accuracy during training but fails in production.

Q3: Why is leakage dangerous?

A: It creates an illusion of high accuracy. The model appears to be 99.9% accurate, but it is just performing a database lookup of the leaked column. In a live system, the leaked column is absent, and the model's actual accuracy collapses.

Q4: What is target leakage?

A: A specific form of leakage where the target variable (the class label) is present in the input feature set, either directly or through a highly correlated proxy variable.

Q5: Why was your previous accuracy misleading?

A: Sequential test data slicing gave Gateway, Fog, and Cloud test splits containing 100% normal samples. A model that predicted 'normal' 100% of the time achieved 99% accuracy while having a recall of 0% for attack detection. This is a classic 'Zero Support' class imbalance trap.

Q6: Why StratifiedKFold?

A: It guarantees that each partition (B.1, B.2, B.3) maintains the exact same ratio of normal and attack samples as the parent dataset (~8.4% attacks). This ensures that evaluation support is non-zero and consistent.

Q7: What happens with sequential splitting?

A: In sorted network logs, all normal packets sit at the top and attacks are at the bottom. Sequential slicing causes early splits to receive 0% attacks, and late splits to receive 100% attacks, introducing extreme test bias.

Q8: Why normalize the data?

A: Network features like byte counts range in the millions, while port flags range from 0 to 1. Without scaling, features with massive numeric ranges dominate the gradient updates, preventing convergence.

Q9: Why SelectKBest?

A: It uses univariate statistical tests to select the top K features most strongly correlated with the target, removing irrelevant features and reducing computation overhead.

Q10: Why VarianceThreshold?

A: It removes zero-variance features (columns where every single row has the same value). These columns contain no statistical information and waste model weights.

5. Engineering Decisions & Parameters

Q1: Why float16?

A: Casting model weights from 32-bit floats to 16-bit half-precision (`float16`) reduces the parameter payload size by approximately 50%, saving bandwidth over cellular/satellite IoT connection links.

Q2: Why convert back to float32?

A: PyTorch's mathematical operations (like FedAvg tensor addition and division) are prone to underflow or overflow instability in float16. We cast back to float32 before aggregation to preserve numerical stability.

Q3: Why 5 epochs?

A: Through empirical testing, local losses converged stably at 5 epochs (Sensor 1 loss: 0.6425, Sensor 2: 0.6563). Fewer epochs caused underfitting, while more epochs wasted computational resources at edge devices.

Q4: Why Mean + 4*Std?

A: Network traffic exhibits natural bursts and spikes. A relaxed boundary of 4 standard deviations accommodates normal flow variance, clearing out false positives (1,504 under 4-sigma vs. 3,817 under 3-sigma).

Q5: Why not Mean + 3*Std?

A: A 3-sigma threshold is too restrictive. It flags normal high-traffic bursts as anomalies, resulting in a high false alarm rate and low precision on benign data.

Q6: Why SHAP?

A: SHAP (SHapley Additive exPlanations) is based on cooperative game theory. It provides mathematically consistent feature attribution values, ensuring that features are ranked reliably.

Q7: Why not LIME?

A: LIME builds local surrogate models by perturbing inputs, which makes it highly unstable (can give different explanations for the same input between runs). SHAP provides globally consistent additive values.

Q8: Why remove the classifier head?

A: The legacy hybrid design had a classifier head to output logits. By focusing strictly on unsupervised anomaly detection, we evaluate anomalies via reconstruction error MSE, making the classification head redundant.

Q9: Why only reconstruct sequences?

A: Removing the classification head reduced the model parameter footprint, making weights more lightweight for decentralized transfers and focusing the model's capacity on reconstruction.

Q10: Why sequence length = 10?

A: A sequence length of 10 provides enough historical context to capture temporal flow dynamics while keeping the input dimensions small enough to fit in low-memory edge devices.

6. Debugging & Troubleshooting

Q1: Tell me about the biggest bug you fixed.

A: The biggest bug was the target leakage from the `Attack_categories` column. The encoder cheat-memorized the class label via the one-hot encoded proxy column. Fixing it required restructuring the preprocessing pipeline to explicitly isolate `Label` and drop `Attack_categories` from features.

Q2: What was the Gateway overwrite issue?

A: The Gateway class had a single instance variable `received_weights`. When Edge 1 and Edge 2 uploaded their weights asynchronously, the second upload overwrote the first, meaning only Edge 2's weights reached the Fog node for aggregation.

Q3: How did you find it?

A: I printed the weights dictionary keys at the Gateway during aggregation and noticed only one client's weight dict was present, and the Fog aggregated loss matched Sensor 2's training loss exactly.

Q4: How long did it take to resolve?

A: It took one troubleshooting cycle to isolate the reference overwrite and refactor Gateway `receive_data` to store weights in a dictionary keyed by node ID.

Q5: How did you verify the fix?

A: I logged the length of the Gateway's weights list, verifying it contained exactly two weight dictionaries before performing the average aggregation loop.

Q6: Which bug taught you the most?

A: The target leakage bug. It taught me that high model accuracy (e.g. 99.9%) is often a sign of data leakage or evaluation errors rather than a perfect model. Debugging the pipeline is as critical as debugging the model.

Q7: What was the hardest part of the project?

A: Designing the sequence alignment window. Network packet records are continuous. Aligning sequence batches of shape (batch, 10, features) with target vectors corresponding to the label of the *last* packet in the sequence required precise indexing to avoid off-by-one errors.

7. Evaluation & Metrics

Q1: Why is your precision only around 22%?

A: The model operates in an unsupervised anomaly detection paradigm with an imbalanced test split (~8.4% attacks). Because normal packets outnumber attacks 12:1, even a small false-positive rate (11% normal packets incorrectly flagged) yields a high absolute number of false positives (1,504 normal vs. 438 true attacks caught). This mathematically limits precision to 22.5%, representing the standard precision-recall trade-off.

Q2: Is 84% accuracy good?

A: Yes, for unsupervised reconstruction models. It represents a realistic, leak-free evaluation on a stratified dataset, capturing 36% of attacks while maintaining 89% recall on normal traffic.

Q3: Why did accuracy decrease?

A: In early tests, accuracy was 99% due to target leakage and sequential split issues. Once the target leak was resolved and test support was stratified, the accuracy adjusted to a realistic, leak-free 84.5%.

Q4: Would you rather have higher recall or higher precision?

A: In network intrusion detection, **Recall** is prioritized. Missing an attack (False Negative) can result in a breach. A False Positive (low precision) creates a false alarm that can be filtered by secondary logs.

Q5: Explain your confusion matrix.

A: In the final Cloud evaluation: TN = 11,777 (normal packets correctly identified), FP = 1,504 (normal flagged as attack), FN = 784 (attacks missed), TP = 438 (attacks correctly caught).

Q6: Why did you choose those metrics?

A: Accuracy evaluates overall correct classifications. Precision, Recall, and F1 measure performance specifically on the minority attack class.

Q7: How do you know the model isn't overfitting?

A: Edge devices train only on benign traffic from Part A, while testing is conducted on entirely unseen stratified partitions from Part B, ensuring the model generalizes to new flows.

Q8: Why balanced test splits?

A: Stratifying test splits (B.1, B.2, B.3) ensures that evaluation support is identical across layers, preventing deceptive metrics caused by imbalanced evaluation sets.

8. Explainability & Interpretation

Q1: What is and why SHAP?

A: SHAP (SHapley Additive exPlanations) is a model explanation tool. We use it to explain the reconstruction MSE of the Mini Transformer Autoencoder, making its anomaly classifications interpretable.

Q2: What does SHAP actually tell us?

A: It computes Shapley attribution values for each input feature. A positive value indicates the feature increased the reconstruction error (pushing the sequence toward an anomaly classification).

Q3: Can SHAP be trusted?

A: Yes. SHAP is based on cooperative game theory axioms, guaranteeing mathematical consistency and local accuracy that heuristic explainers lack.

Q4: What feature contributed the most?

A: Features related to network addresses, port mappings, flag states, and byte lengths show the highest attributions during reconstruction anomalies.

Q5: How is SHAP helping an analyst?

A: A security analyst receives an alert alongside a SHAP chart explaining *why* the flow was flagged (e.g. source port spikes), reducing diagnostic time from hours to seconds.

9. Optimization, Future Work & Code Implementation

Q1: Why float16 and what performance did it provide?

A: It reduces the weight dictionary payload size by approximately 50%, halving bandwidth costs on edge channels.

Q2: Would INT8 be better?

A: INT8 reduces payload sizes by 75%, but can lead to quantization error and loss of precision on continuous features. It would require quantization-aware training (QAT) to implement successfully.

Q3: What other optimizations would you make?

A: Implement model pruning (removing redundant attention weights) and communication compression (gradient sparsification).

Q4: What would you improve if given another month?

A: Integrate homomorphic encryption for secure aggregation, implement multi-round federated training, and benchmark the model on real-world edge hardware like a Raspberry Pi.

Q5: How would you deploy this in production?

A: Compile the PyTorch model to ONNX format, deploy ONNX Runtime on edge nodes, and ingestion network packet flows via Kafka message brokers.

Q6: Write code to compute reconstruction error and detect anomalies.

```
import torch
import torch.nn as nn

def detect_anomalies(model, dataloader, threshold, device):
    model.eval()
    criterion = nn.MSELoss(reduction='none')
    predictions = []

    with torch.no_grad():
        for X, y in dataloader:
            X = X.to(device)
            reconstructed = model(X)

            # Calculate MSE per sequence (mean over seq_len and input_dim)
            mse = criterion(reconstructed, X).mean(dim=[1, 2])
            preds = (mse > threshold).long()
            predictions.extend(preds.cpu().numpy())

    return predictions
```

Q7: Implement the Federated Averaging (FedAvg) aggregation loop in Python.

```
import copy

def federated_averaging(client_weights_list):
    # Assumes client weights are in a list of PyTorch state_dicts
    global_weights = copy.deepcopy(client_weights_list[0])
    num_clients = len(client_weights_list)

    for key in global_weights.keys():
        # Sum weights from all clients for the parameter key
        for i in range(1, num_clients):
            global_weights[key] += client_weights_list[i][key]

        # Average the parameters
        global_weights[key] = global_weights[key] / num_clients

    return global_weights
```

Q8: If accuracy dropped, why do you say the project improved?

A: High accuracy driven by target leakage is a system flaw. Correcting target leakage dropped accuracy to a realistic, leak-free 84.5%. This makes the model robust, generalizable, and mathematically valid for real-world deployment.